

```

1  #include <stdio.h>
2  #define MAX_Length 20
3
4  typedef struct
5  {
6      /* data */
7      int u, v; // u = dinh 1, v = dinh 2
8  }edge;
9
10 typedef struct
11 {
12     /* data */
13     int n, m; // n = so dinh, m = so canh
14     edge data[MAX_Length]; // e = tap hop cac canh
15 }graph;
16
17 typedef struct
18 {
19     int data[MAX_Length];
20     int size;
21 }list;
22
23 void makeNull(list *L){
24     L->size = 0;
25 }
26 int elementAt(list *L, int x){
27     return L->data[x];
28 }
29 void pushList(list *L, int x){
30     L->data[L->size] = x;
31     L->size++;
32 }
33
34 void initGraph(graph *G, int n){
35     G->n = n;
36     G->m = 0;
37     printf("Do thi G duoc khoi tao voi so dinh n = %d va so canh m = %d\n", G->n, G->m);
38 }
39
40 void addEdge(graph *G, int u, int v){
41     int i;
42     for(i = 0; i < G->m; i++){
43         if ((G->data[i].u == u && G->data[i].v == v) || (G->data[i].u == v && G->data[i].v == u))
44         {
45             printf("Canh da co trong do thi!");
46             return;
47         }
48     }
49     G->data[G->m].u = u;
50     G->data[G->m].v = v;
51     G->m++;
52     printf("Do thi da them vao canh %d %d\n", G->data[G->m-1].u, G->data[G->m-1].v);
53 }
54 int adjacent(graph *G, int u, int v){
55     int i;
56     for(i = 0; i <= G->m; i++){
57         if ((G->data[i].u == u && G->data[i].v == v) || (G->data[i].v == u && G->data[i].u == v))
58         {
59             return 1;
60         }
61     }
62     return 0;
63 }
64
65 void degree(graph *G, int x){
66     int i, count = 0;
67     for(i = 0; i <= G->m; i++){
68         if (adjacent(G, x, i))
69         {
70             count++;
71         }
72     }
73     printf("Bac cua canh %d la: %d\n", x, count);
74 }

```

```

76 list neighbors(graph *G, int x){
77     list listVertex;
78     makeNull(&listVertex);
79     int i;
80     for(i = 1; i<=G->n; i++){
81         if (adjacent(G, x, i) == 1)
82             {
83                 pushList(&listVertex, i);
84             }
85     }
86     return listVertex;
87 }
88
89 // DFS de quy
90
91 int mark[100];
92
93 void traversal(graph *G, int x) {
94     if(mark[x] == 1) return;
95     printf("Duyet %d\n", x);
96     mark[x] = 1;
97
98     list ds = neighbors(G, x);
99     int j;
100     for (j = 0; j< ds.size; j++){
101         int e = elementAt(&ds, j);
102         traversal(G, e);
103     }
104 }
105
106 void depth_first_search(graph *G) {
107     int i;
108     for(i = 1; i<= G->n; i++)
109         mark[i] = 0;
110     traversal(G, 1);
111 }
112
113 // STRONG CONNECT
114
115 typedef struct {
116     int data[100];
117     int size;
118 }Stack;
119
120 void make_null_stack(Stack *S) {
121     S->size = 0;
122 }
123
124 void push(Stack *S, int x) {
125     S->data[S->size] = x;
126     S->size++;
127 }
128
129 void pop(Stack *S) {
130     S->size--;
131 }
132
133 int top(Stack S) {
134     return S.data[S.size-1];
135 }
136
137 int empty(Stack S) {
138     return S.size == 0;
139 }
140
141
142 #define min(a,b) ((a)>(b))? b: a
143
144
145 int num[100], min_num[100], on_stack[100], k;
146 Stack S;
147
148

```

```

149 void strong_connect(graph *G, int x) {
150     num[x] = k;
151     min_num[x]=k;
152     k++;
153     push(&S, x);
154     on_stack[x] = 1;
155
156     list ds = neighbors(G, x);
157     int j;
158
159     for(j = 0; j< ds.size; j++) {
160         int e = elementAt(&ds, j);
161         if(num[e] < 0) {
162             strong_connect(G, e);
163             min_num[x] = min(min_num[x], min_num[e]);
164         } else
165             if(on_stack[e])
166             {
167                 min_num[x] = min(min_num[x], num[e]);
168             }
169
170     }
171     printf("min_num[%d] = %d\n", x, min_num[x]);
172
173     if(num[x] == min_num[x]) {
174         printf("%d là đỉnh khop\n", x);
175         int w;
176         do{
177             w = top(S);
178             pop(&S);
179             on_stack[w] = 0;
180         }while(w!=x);
181     }
182 }
183
184
185
186 int main(int argc, char const *argv[])
187 {
188     freopen("dothi.txt", "r", stdin);
189     int n, m, u, v, i, j;
190     scanf("%d%d", &n, &m);
191     graph G;
192     initGraph(&G, n);
193     for(i = 1; i<=m; i++){
194         scanf("%d%d", &u, &v);
195         addEdge(&G, u, v);
196     }
197
198     depth_first_search(&G);
199
200     // strong_connect
201
202     for(v = 1; v<=n; v++) {
203         num[v] = -1;
204         on_stack[v] = 0;
205     }
206
207     k = 1;
208     make_null_stack(&S);
209
210     for(v = 1; v<= n; v++){
211         if(num[v] == -1)
212             strong_connect(&G, v);
213     }
214
215     return 0;
216 }

```